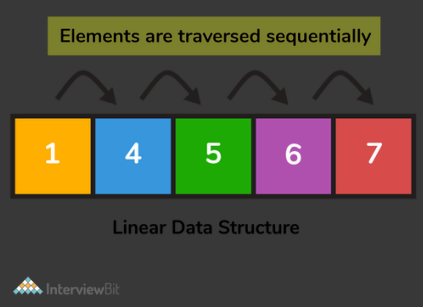
SUMMARIZATION

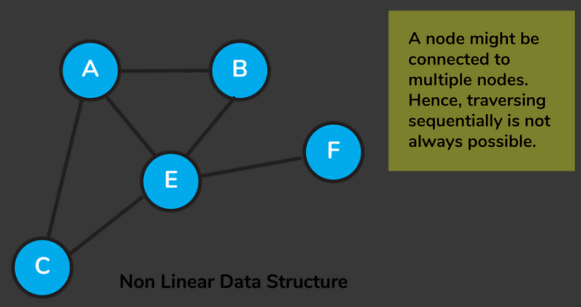
WHAT IS A DATA STRUCTURE AND THE TYPES

The data structure is the data type that is made by the user.

The types:

- **Linear data structure:-** When all the elements are arranged in a linear order.
- **Nonlinear data structure:-** When all the elements are not arranged in a linear order.
- **Static data structure:-** The memory is allocated at compile time; therefore, the maximum size is fixed.
  Fast access, slower insertion and deletion.
- **Dynamic data structure:** The memory is allocated at run time; therefore, the maximum size is flexible.
  Faster insertion and deletion, slower access.





PASSING STRUCTURE TO A FUNCTION.

There are three ways to pass a structure to a function:

- **Passing the entire structure (By Value):** The function receives a copy of the structure.
  Changes inside the function do not affect the original variable.
  EX: **void print(Pet p);**

- **Passing the structure's address (By Reference):** The function receives a pointer to the structure.
  This is more memory-efficient and allows the function to modify the original data using the -> operator.
  EX: **void initialize(Pet *p);**
- **Passing individual members:** Passing only specific variables from the structure using the dot . operator.
  EX:**void Age(int pet.age);**

SIZE OF STRUCTURE.

- The size of a structure is not always the simple sum of its members' sizes.
  It is governed by a few key factors:
- **Sum of Members:** Primarily, the size depends on the data types used inside the struct (int, char, float).
- **Member Order:** The sequence in which variables are declared affects how the compiler arranges them in memory.
- **Compiler Architecture:** The size can vary between a 32-bit and a 64-bit system, especially for pointers.
- **The sizeof() Operator:** In C, we use the sizeof operator to find the actual size of a structure in bytes, which includes any extra space added by the compiler.

MEMORY PADDING , ALIGNED MEMORY AND UNALIGNED MEMORY

Memory Padding

- It is the process where the compiler inserts empty bytes between structure members or at the end of the structure.
- **Purpose:** To ensure that each member starts at a natural alignment address, which makes it easier for the processor to read the data.

Feature	Aligned Memory	Unaligned Memory
Storage Rule	Variables are stored at addresses divisible by their size a 4-byte int starts at address 0, 4, 8.	Variables are stored at any available address without considering their size or boundaries.
Processor Access	The processor reads memory in 4 bytes at a time. Data is fetched in a single cycle.	The processor might need multiple cycles to fetch a single variable that spans across two memory words;
Efficiency	**High efficiency:** Faster execution and better performance.	**Low efficiency:** Slower execution due to extra CPU cycles and bit-shifting.
Memory Usage	Uses more memory because of the added Padding bytes.	Uses less memory as it packs data tightly without gaps.
Time Complexity	Minimal time to access data.	More time is required to access and process the data.

THE DIFFERENCE BETWEEN THE STRUCTURES AND OBJECT

- The Structure is a user-defined data type.
- It acts as a template that defines what kind of data will be stored.
- Defining a structure does not reserve any memory space; it only tells the compiler how the data will look.

The Object

- The Object is the actual variable created using that structure.
- It is the representation of the structure that holds your specific information.
- Memory is allocated only when an object is created.